Ooty Radio Telescope: Declination Control System - Reverse Engineering, Custom Hardware Monitoring, and FPGA Prototyping

Jayanth Krish
School of Electrical Engineering
Manipal Institute of Technology, MAHE
jayanth.mitmpl2024@learner.manipal.edu
Manipal, India

Sameer R Bharadwaj
Radio Astronomy Centre
NCRA-TIFR
sameer@ncra.tifr.res.in
Ooty, India

Abstract

The Ooty Radio Telescope (ORT) utilizes a legacy Declination (DEC) Control System to achieve North-South beam steering via microstrip phase delay elements. Driven by an aging IBM PC/XT/AT architecture and Intel 8273 Peripheral controllers, modernizing this critical infrastructure requires a definitive understanding of its data-link protocols and physical layer behaviors. This report details the reverse engineering, custom hardware monitoring, and RTL prototyping undertaken to aid efforts in transitioning the Master Control Room (MCR) to a modern solution. First, legacy control software (DECMAY19.C) was analyzed to map command structures, including PWR ON/OFF and RF/IF DEC frames and the telemetry frames received from the field modules. To capture and validate physical transmissions, a custom hardware sniffer was developed using a TI C2000 microcontroller. Asynchronous clock recovery challenges on the physical line was overcome and captured SDLC frames were decoded using custom Python tooling, which successfully mapped the system's specific NRZI encoding behavior and CRC-CCITT implementation. Finally, these verified dataframes were utilized to design a modern SystemVerilog SDLC controller. Simulation of the RTL modules: including bit stuffing, CRC implementation, frame generation, and NRZI coding was successfully completed, establishing a verified pathway for an FPGA-Based modernization of the ORT DEC interface.



Figure 1: Ooty Radio Telescope

(2026) Ooty Radio Telescope: Declination Control System - Reverse Engineering, Custom Hardware Monitoring, and FPGA Prototyping

1 Introduction & Objectives

1.1 System Overview

The Ooty Radio Telescope (ORT) relies on a legacy Declination (DEC) Control System to achieve electronic North-South (N-S) beam steering. This beam steering is executed by introducing phase delays across the antenna array using discrete microstrip delay elements.

This operation is controlled by the Master Control Room (MCR), which communicates with distributed field modules (Set of 3 Slave Cards) to set bit configurations and the slave cards respond with telemetry data with information on whether the bits are set properly. However, the current infrastructure is built on aging IBM PC/XT/AT architecture, relying on obsolete and End of Life (EoL) integrated circuits(Ex: Intel 8273, Intel 8254 etc.) and undocumented software running on the Intel 8085s in the field modules. To ensure the long-term reliability of the ORT, this communication interface must be understood and modernized.

1.2 Project Goals

The primary objective of this internship was to reverse-engineer the legacy MCR communication system and establish a pathway for modernizing the system. To achieve this, the project was broken down into four key deliverables:

1. **Legacy System Analysis:** Deconstruct the existing control software (`DECMAY19.C` and `all_1.C`) to map operation modes, addressing schemes, and data flow.
2. **Protocol Modeling:** Develop Python-based models of the Synchronous Data Link Control (SDLC) protocol, Non-Return-to-Zero Inverted (NRZI) encoding, and Cyclic Redundancy Check (CRC) behavior.
3. **Hardware Monitoring:** Design and implement a custom hardware sniffer using a TI C2000 microcontroller to physically capture, and log communications to and from the MCR on the wire.
4. **FPGA Prototyping:** Utilize the captured and verified dataframes to design, simulate, and verify a custom SDLC controller as a proof-of-concept for future modernization.

2 Legacy System

The physical layer and data transmission of the legacy DEC system are structured across three stages: the Master Control Room (MCR) interface, the isolation barrier, and the 3 field cards (New Array Dec. Control & Monitor System).

The MCR Interface and Isolation: At the MCR, communications are driven by an IBM PC/XT/AT add-on card equipped with an Intel 8273 SDLC Controller operating in *asynchronous mode*. On this add-on card, the outgoing transmit (Tx) signals are converted into differential pairs, while incoming receive (Rx) signals are converted back to single-ended logic.

These signals are routed via a DB9 connector into a dedicated Galvanic Isolation Box. Inside this enclosure, optocouplers provide electrical isolation to protect the MCR hardware from ground loops and electrical transients across the telescope array. The isolation box handles subsequent single-ended-to-differential conversions, outputting a four-wire communication bus (Tx+, Tx-, Rx+, Rx-) that connects to all field modules in a multidrop bus configuration.

Field Module Slave Cards: Out on the telescope array, the multidrop bus interfaces with the field modules, which contain designated Slave Cards (Cards 1, 2, and 3). The primary slave card has an Intel 8085 Microprocessor paired with an onboard Intel 8273 controller to decode the SDLC communications.

The slave cards house the physical control interfaces for the delay modules. They feature digital latches that assert the appropriate control bits to the microstrip delay cards to set the phase delays. Additionally, the cards integrate analog multiplexers and Analog-to-Digital Converter (ADC) interfaces to measure local control voltages, which are packaged and transmitted back to the MCR as system telemetry.

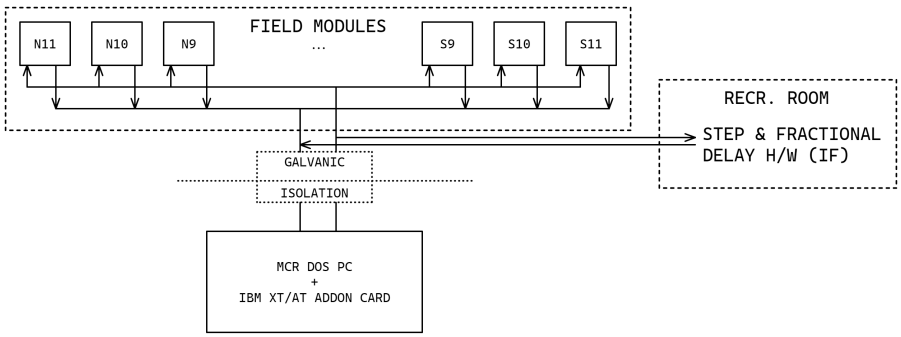


Figure 2: Declination Control System - Simplified Block Diagram

3 SDLC Protocol & NRZI Encoding - Data Link + PHY Layer Discussion

To effectively monitor and eventually replace the legacy Intel 8273 controller, the mechanics of the Synchronous Data Link Control (SDLC) protocol and its physical layer encoding had to be modeled and verified against raw line data.

3.1 SDLC/NRZI Fundamentals

Communications between the Master Control Room (MCR) and the field modules are encapsulated in standard SDLC frames. The physical transmission relies on two fundamental data-link mechanisms:

- **NRZI Encoding:** The physical line utilizes Non-Return-to-Zero Inverted (NRZI) encoding. In this scheme, a logical `0` is represented by a state transition on the wire, while a logical `1` results in no transition. This ensures sufficient transitions for clock recovery during long sequences of zeros.
- **Bit Stuffing:** Frame boundaries are marked by a specific Flag byte (`0x7E`, or `01111110` in binary). To prevent payload data from accidentally simulating this flag and prematurely terminating the frame, a transmitter inserts (stuffs) a logical `0` after any sequence of five consecutive `1`s in the payload. The receiver must strip (unstuff) this `0` to recover the original data.

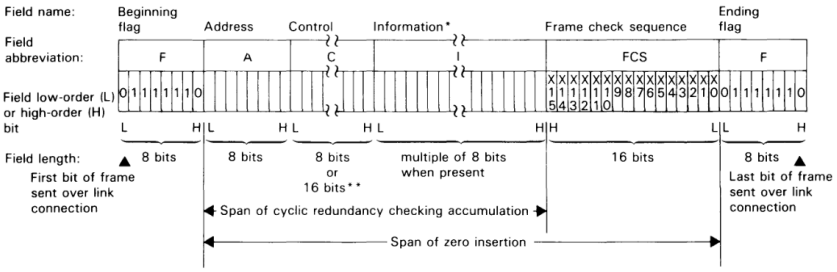


Figure 3: Bit Order - Frame Transmission

3.2 CRC Implementation

Data integrity within the SDLC frame is verified using a 16-bit Frame Check Sequence (FCS) located immediately before the closing flag. The legacy system utilizes the **CRC-CCITT** standard, defined by the generator polynomial:

$$x^{16} + x^{12} + x^5 + 1$$

In standard hexadecimal representation, this polynomial is `0x1021`. However, note the following nuance in the SDLC transmission that requires attention in computation of the FCS Field to decode the physical line data:

- **Payload Transmission:** The Address, Control, and Information fields are transmitted onto the physical wire **LSB-first**.
- **FCS Transmission:** The calculated FCS field is transmitted onto the wire **MSB-first**

To handle the sequential LSB-first data arriving from the hardware monitor, a reversed polynomial (`0x8408`) was utilized. This shortcut allows the sequential bit-stream to be fed directly into the CRC shift register as it is captured without requiring bit-reversal operations on every byte. Because the resulting CRC value inherently holds its LSB in the leftmost position, a final bit-reversal of the 16-bit result perfectly matches the MSB-first FCS field appended to the end of the transmitted frame.

3.3 Python Tools

Before deploying the hardware monitor, the theoretical protocol definitions were modeled using Python within a Jupyter Notebook environment.

These software tools were developed to mathematically simulate the entire transmission pipeline. Functions were written to take raw hexadecimal payloads, append the reversed-polynomial CRC-CCITT FCS, apply zero-bit stuffing, and encode the final stream into NRZI transition logic.

4 Hardware Monitor Design & Troubleshooting

To capture and verify the legacy SDLC frames directly from the physical wire, a custom hardware monitor was developed using a TI C2000 microcontroller (TMS320F28379D Launchpad XL). The objective was to non-intrusively sniff the multidrop bus and log the raw bitstream to a PC for identifying the data frames and generate test vectors for any custom implementation.

4.1 Overview

The primary challenge in sniffing the SDLC multidrop bus is clock synchronization. The Intel 8273 controller operates asynchronously; it does not provide an explicit clock signal on a separate wire to the receiver. Instead, the receiver must extract the bit clock from the incoming NRZI data stream.

The legacy MCR hardware attempts to solve this by generating an internal bit clock using a Digital Phase-Locked Loop (DPLL). The Intel 8273 outputs this recovered clock via a dedicated DPLL pin as an active-low pulse.

4.2 Initial Challenges

Initial attempts to sample the data line by directly reading this DPLL output pin with the C2000 microcontroller failed. Hardware scoping revealed that the DPLL pulse duration was approximately `300ns` wide. This active-low pulse was too narrow for the C2000's general-purpose inputs to reliably detect and trigger an interrupt consistently, leading to missed bits and corrupted frame captures.

4.3 Hardware Troubleshooting and Final Setup

To overcome the limitations of the narrow DPLL pulse, external hardware conditioning was required before routing the signal to the microcontroller.

A **74LS123 Dual Retriggerable Monostable Multivibrator** was used. The `300ns` DPLL pulse was routed into the 74LS123, which was configured to stretch the pulse duration to approximately `1us`. This stretched, reliable clock signal was then fed into the C2000's External Interrupt (XINT1) pin, ensuring the microcontroller could reliably sample the data line on every valid clock cycle.

Capture Logic: The sniffing software (written in C) operates via a two-stage interrupt mechanism to avoid logging endless idle states:

1. **Frame Start Detection:** The SDLC data line is routed to a secondary external interrupt (XINT2) configured to trigger on a transition. Because every transmission begins with a `0x0000` synchronization preamble (which generates immediate transitions on the NRZI-encoded wire), this interrupt detects the start of a frame.
2. **Bit Sampling:** Once the frame start is detected, the primary interrupt (XINT1) is enabled. It uses the stretched DPLL clock to sample the actual data line sequentially, pushing each bit into an internal memory buffer (`data_buf`).

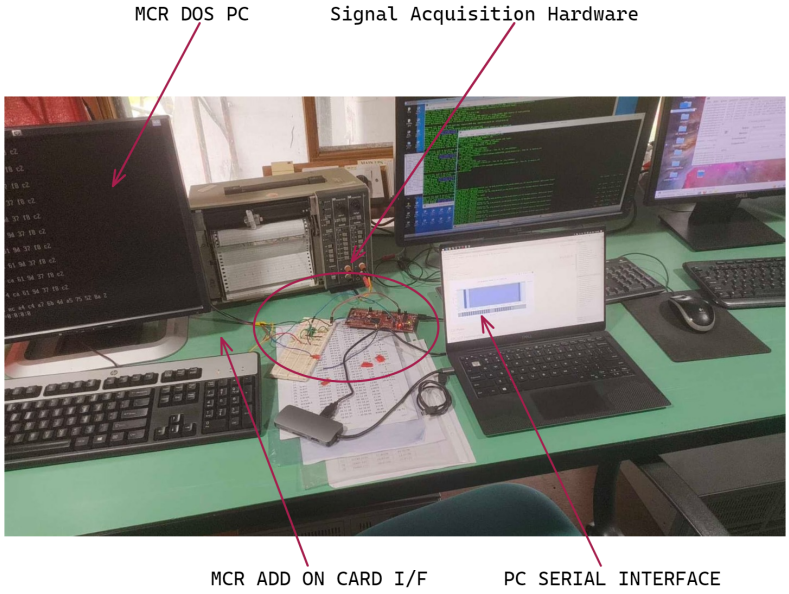


Figure 4: MCR - Frame Capture Setup

4.4 Data Logging and Processing Pipeline

Once a complete frame is captured in the C2000's internal buffer, the data is pushed asynchronously over a UART-to-USB serial connection to a host PC.

The host PC runs a custom Python-based serial monitor designed to detect synchronization headers (`0xAA, 0x55`), identify packet types (TX vs. RX), and save the raw bytestreams to disk with appropriate timestamps.

Post-Processing: To translate these raw binary captures into actionable data, an additional Python environment (Jupyter Notebook) was utilized. This toolset provided two critical functions:

- **Visualization:** The raw bits were formatted into CSV files and graphed using plotting libraries (`yaml2plot/plotly`). This allowed for the visual recreation of the NRZI waveforms, confirming visually where bit-stuffing and frame flags (`0x7E`) occurred.
- **CRC Verification:** The Python environment was used to run the captured payloads through the custom reversed-polynomial (`0x8408`) CRC algorithm. By successfully matching the

Python-calculated CRC against the FCS captured directly off the wire, the physical data integrity of the MCR's transmission was definitively validated.

The completely decoded data frames, mapped to their specific MCR operations, have been tabulated into Excel workbooks. Both these workbooks and the complete Python Notebook are linked in the references.

Through the deployment of the C2000-based hardware monitor, physical data frames were successfully captured, decoded, and analyzed. This allowed for a mapping of the command and telemetry structures that are otherwise obscured within the legacy C code.

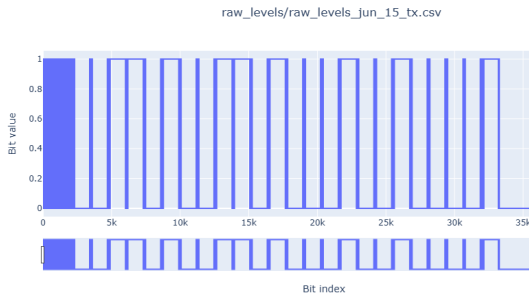


Figure 5: MCR Data Transmission

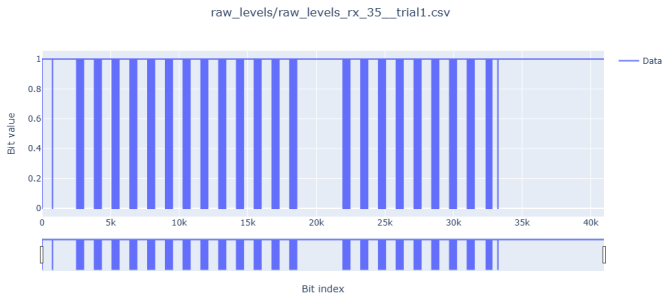


Figure 6: Field Modules Telemetry

5 Data Capture & Observations

(Access `DECMAY19.C` decoded data here: DEC Control & Telemetry Data Frames)

(Access `ALL_AD1.C` data here: Single Module Data Transfer)

5.1 Asynchronous Clock Synchronization

A critical observation across all MCR command transmissions and field module responses is the presence of a synchronization preamble. Every data transmission on the physical line begins with 0x0000.

Because the system uses Non-Return-to-Zero Inverted (NRZI) encoding (where a logical '0' dictates a state transition and a logical '1' dictates no transition) transmitting 0x0000 forces 16 consecutive state transitions on the wire. This dense transition cluster is purposefully designed to synchronize the phase-locked loop (DPLL) bit clock of the receiving SDLC controllers. Incorporating this preamble logic is critical for reliable clock recovery in any future custom implementation. ## MCR Transmission (Command Frames)

The MCR transmissions dictate the operational state of the DEC System. All communications follow the standard Synchronous Data Link Control (SDLC) frame structure, enveloped by standard Flag bytes (F = 0x7E), a 16-bit Frame Check Sequence (FCS), and the Address (A), Control (C), and Information (I) fields.

Through hardware monitoring and protocol decoding, the exact frame structures for the primary MCR commands have been mapped as follows:

5.2 Control Frames

1. System Power Commands: Power commands are broadcast to all modules simultaneously.

- **Power OFF Frame:** Utilizes a broadcast address and contains no Information field.
  Structure: `F || A = 0xFF || C = 0xE0 || I_length = 0 || FCS || F`
- **Power ON Frame:** Utilizes a null address and control byte, relying on a 255-byte I-field (consisting entirely of `0x00`) to initialize the system.

Structure: `F || A = 0x00 || C = 0x00 || I_length = 255B || FCS || F`

2. RF DEC Configuration Frames: These frames dictate the primary RF delay configurations for modules 1 through 22. The Information field consists of 13 bytes, where the lower 6 bits (LSBs) of the 13th byte are dedicated exclusively to setting the Local Oscillator (LO) phase bits.

- **Target Modules:** 1 to 22 (`0x01` to `0x16`)
- **Structure:** `F || A = modnum || C = 0x57 || I (13 bytes) || FCS || F`

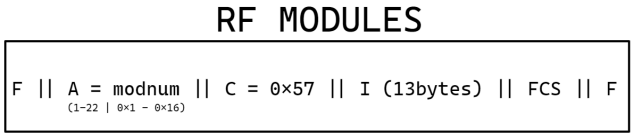


Figure 7: RF MODS - Data Frame

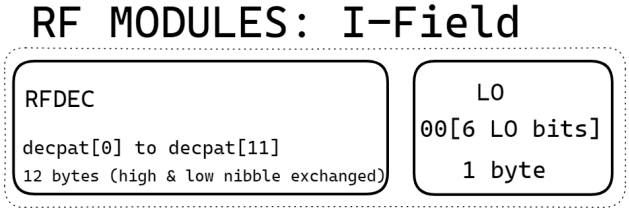


Figure 8: RF MODS - I Field

3. IF DEC Configuration Frames These frames define the Intermediate Frequency configurations and are specifically targeted at Module 23. The Information field is expanded to 21 bytes.

- **Target Module:** 23 (`0x17`)
- **Structure:** `F || A = 0x17 || C = 0x97 || I (21 bytes) || FCS || F`
- **Note on IF I-Field Composition:** The final 6 bytes of this 21-byte payload depend on the operational mode. During a Low DEC Request (normal settings), the last 6 bytes are hardcoded to `0x51, 0x55, 0x55, 0x52, 0x55, 0x55`. If the `HIGDEC` flag is active, the system dynamically overwrites these final 6 bytes with a specific high-declination pattern (`hd_pattern`) calculated by the control software.

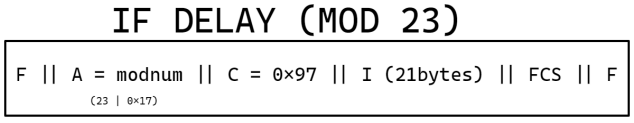


Figure 9: IF DELAY BITS - Data Frame

5.3 Telemetry Data Reception

The MCR PC receives telemetry data from the field slave cards to know whether the delay configuration bits were set successfully and the current state of the field module. The hardware sniffer successfully captured and decoded these incoming frames.

All observed telemetry frames utilize a standard header configuration:

- **Address Byte:** `0xFF`
- **Control Byte:** Observed as either `0x57` (RF MODULES) or `0x97` (IF MODULE) depending on the module.

Payload Structure and Bit Interleaving The payloads are structured to provide both the measured data state and a corresponding hardware error flag for each bit.

- **RF Module Telemetry:** 50-byte payload (representing 25 data bytes and 25 error-flag bytes).
- **IF Module Telemetry:** 42-byte payload (representing 21 data bytes and 21 error-flag bytes)

To interpret the telemetry data, the telemetry parsing function (`chk_rx_data`) from the legacy C code was understood. The analysis revealed that the data and error flags are not sent as sequential blocks; rather, they use a bit-interleaving scheme. The software parses the incoming buffer in two-byte pairs, applying specific bitmasks to separate the error flags from the telemetry data:

- **Bad Bit Mask (`0x01, 0x04, 0x10, 0x40`):** Extracts bits 0, 2, 4, and 6 to reconstruct the hardware error state.
- **Data Mask (`0x02, 0x08, 0x20, 0x80`):** Extracts bits 1, 3, 5, and 7 to reconstruct the actual data state.

By de-interleaving this data, the MCR calculates malfunctioning or unresponsive delay elements. The resulting "bad bits" and "bad data" are actively logged to the `badbits.dat` file and printed on the console.

5.4 Note on the LO Phase Bits

Hardware monitoring of the RF DEC command frames confirmed that the 6 Local Oscillator (LO) Phase Bits (located in the lower bits of the 13th byte) are Declination (DEC) dependent.

By analyzing the legacy control software (`ifdec` and `ifdec_high` functions), it was determined that the final LO phase state for each module is continually updated and derived from two distinct components:

- **Static LO Phasing (Calibration LUT):** A baseline phase value is assigned to each module using a Look-Up Table (`phase.val`). This file is generated during routine LO phasing calibrations to account for inherent, static hardware offsets across the array.
- **Dynamic Residual Phase Correction:** As the beam steering angle changes, the system computes the necessary signal delays. To account for the residuals caused by the physical separation of the Frontend Modules. The software dynamically calculates this difference and updates the LO bits to correct the residual phase, applying the calculated value to the North modules and its exact mathematical complement to the corresponding South modules.

Any future implementation will need to interface with a host system capable of performing these dynamic calculations before framing the SDLC transmission.

For exact implementation in `DECMAY19.C`:

Lines `289-299`: `PHASE.VAL` from LO Phasing loaded

Lines `1264-1440` & `1576-1812` : `ifdec()` & `ifdec_high()` Dec dependent LO Correction

5.5 Note on IF Delays Telemetry (Module 23)

During the hardware monitoring, specific anomalies were identified in the telemetry response specifically from the Intermediate Frequency (IF) module (Module 23).

Telemetry Anomaly: The `0xAA` Payload A consistent observation during nominal operation was the reception of a 42-byte IF payload consisting entirely of `0xAA` bytes. When the legacy parsing logic applies its masks to an `0xAA` (`10101010` in binary) byte:

- Every "Bad Bit" evaluates to `0` (interpreting zero hardware errors).
- Every "Data Bit" evaluates to `1` (interpreting all monitored elements as being in a high state).

Because the physically monitored data states at the module inherently consist of a dynamic mix of `0`s and `1`s depending on the active DEC setting, a static payload evaluating to all `1`s for data is an

anomaly. This results in recurring errors regarding bad bits and data regarding the IF module logged during operation. This consistent `0xAA` readout strongly indicates an unexpected behavior or fault within the telemetry reporting itself, such as a stuck bus, a floating data line, or an ADC interface failure, that requires further investigation.

6 SDLC Controller: (SystemVerilog) Proof of Concept

6.1 Microarchitecture Overview

6.1.1 1. NRZI Encoder (`nrzi_encode.sv`)

SEQUENTIAL LOGIC - **EVENT: Reset** (`rst`) `nrzi_out` = 1 (Line idles high) - **EVENT: Valid Input Bit** (`en`) *If `d_in` = 0:* Toggle `nrzi_out` (`~nrzi_out`) *If `d_in` = 1:* Hold `nrzi_out` at its current state

6.1.2 2. NRZI Decoder (`nrzi_decode.sv`)

SEQUENTIAL LOGIC

- **EVENT: Reset** (`rst`) `prev_nrzi` = 1 (Assume idle state) `d_out` = 1 (Default idle output)
- **EVENT: Sample Strobe** (`en`) *If `nrzi_in` == `prev_nrzi`:* `d_out` = 1 (Line held its state) *If `nrzi_in` != `prev_nrzi`:* `d_out` = 0 (Line toggled) `prev_nrzi <- nrzi_in` (Save current physical bit to compare against the next one) ### 3. RX Bit Unstuffer (`rx_bit_unstuff.sv`)

SEQUENTIAL LOGIC *(Note: `en_out`, `flag_detect`, and `abort_detect` always default to 0 at the start of every cycle to create single-cycle pulses).*

- **EVENT: Reset** (`rst`) `ones_count` = 0 `d_out` = 1
- **EVENT: Recovered Bit Arrives** (`en_in`) *If `d_in` = 1:* `ones_count` += 1 (Clamped at 15 to prevent integer wrap-around during long idles) *If `ones_count` >= 6:* `abort_detect` = 1 (7+ ones is an SDLC Abort sequence) *Else:* `en_out` = 1, `d_out` = 1 (Pass the valid 1 to the FSM) *If `d_in` = 0: If `ones_count` = 5:* **Stuffed Zero.** Drop it entirely (`en_out` remains 0). *If `ones_count` = 6:* **Flag Detected.** `flag_detect` = 1. (Do not pass to FSM). *Else:* **Normal Zero.** `en_out` = 1, `d_out` = 0. `ones_count` = 0 (Always reset the counter when a zero is seen)

6.1.3 4. TX Bit Stuffer (`tx_bit_stuff.sv`)

COMBINATIONAL LOGIC

- `stuff_next` = 1 *ONLY IF* `ones_count` == 5.
- `ready` = `~stuff_next` (Tells the upstream TX FSM to freeze and hold its data while the stuffer commandeers the wire to insert a zero).

SEQUENTIAL LOGIC

- **EVENT: Reset** (`rst`) `ones_count` = 0 `en_out` = 0 `d_out` = 1
- **DEFAULT (Every Clock Cycle):** *If `stuff_next` == 1 (Priority):* `en_out` = 1 `d_out` = 0 (Force the stuffed zero out) `ones_count` = 0 (Reset the consecutive counter) *Else if `en_in` == 1 (Normal Data):* `en_out` = 1 `d_out` = `d_in` (Pass the FSM's bit through) *If `d_in` = 1:* `ones_count` += 1 *Else:* `ones_count` = 0 *Else:* `en_out` = 0 (Pipeline is idle, no valid data moving)

6.1.4 5. CRC CCITT FCS (`crc_ccitt_fcs.sv`)

COMBINATIONAL LOGIC

- `feedback` = `data_bit` XOR `crc_reg[0]` (Calculates polynomial alignment for LSB-first transmission).

- `crc_out` = `~crc_reg` (Inverts the final CRC register per the SDLC specification before appending to TX).
- `crc_valid` = (`crc_reg` == `16'hF0B8`) (Checks if the running math exactly matches the CCITT magic residue).

SEQUENTIAL LOGIC - **EVENT: Reset (`rst`)** `crc_reg` = `16'hFFFF` (SDLC requires initializing the register to all 1s, not 0s). - **EVENT: Valid Data Bit** (`en`) *If `feedback` = 1:* `crc_reg` is shifted right by 1, then XOR'd with the reversed CCITT polynomial `16'h8408`. *If `feedback` = 0:* `crc_reg` is simply shifted right by 1.

6.1.5 6. TX Framer (`sdlc_tx_framer.sv`)

SEQUENTIAL LOGIC (The Main FSM) *(Note: `tx_ready` always defaults to 0 at the start of every cycle to create single-cycle pulses).*

- **IDLE** Waits for `tx_valid` asserted by CPU. `tx_data[7:0]` is copied into `shift_reg`. `is_last` <- `tx_last`. `tx_ready` = 1 (Acknowledge byte consumption). `bit_cnt` = 0. `state` -> `TX_OPEN_FLAG`.
- **TX_OPEN_FLAG** *If `bit_cnt` = 7:* `state` -> `TX_PAYLOAD`, `bit_cnt` = 0. *Else:* `bit_cnt` += 1.
- **TX_PAYLOAD** *Only advances if `stuffer_ready` == 1 If `bit_cnt` = 7 (Full Byte Processed): If `is_last` == 1:* `state` -> `TX_FCS`, `bit_cnt` = 0. *Else if `tx_valid` == 1:* Load next byte into `shift_reg`, `is_last` <- `tx_last`, `tx_ready` = 1, `bit_cnt` = 0. *Else:* `state` -> `IDLE` (TX Underflow error condition—no valid data provided in time). *Else:* `bit_cnt` += 1.
- **TX_FCS** *Only advances if `stuffer_ready` == 1 If `bit_cnt` = 15 (2 Bytes Processed):* `state` -> `TX_CLOSE_FLAG`, `bit_cnt` = 0. *Else:* `bit_cnt` += 1.
- **TX_CLOSE_FLAG** *If `bit_cnt` = 7:* `state` -> `IDLE`. *Else:* `bit_cnt` += 1.

COMBINATIONAL LOGIC (Internal Routing Multiplexer)

- **TX_OPEN_FLAG / TX_CLOSE_FLAG** `flag_en_comb` = 1 `flag_d_comb` = `0x7E` bit shifted right by `bit_cnt` (LSB first).
- **TX_PAYLOAD** `stuffer_en_in` = 1 `stuffer_d_in` = `shift_reg` bit shifted right by `bit_cnt` (LSB first). `crc_en` = `stuffer_ready` (Only feed the CRC math engine on the exact cycle the stuffer ingests the bit).
- **TX_FCS** `stuffer_en_in` = 1 `stuffer_d_in` = `crc_out` bit shifted right by `bit_cnt` (LSB first—matches the physical register bit 0 of the right-shifting LFSR). `crc_en` = 0 (Freeze the CRC math engine while we transmit its final value).
- **Global / Default Routing:** `crc_rst` = (`state == IDLE`) (Hold CRC in reset whenever not actively processing a frame).

PIPELINING (1-Stage Stuffer Alignment)

- To match the 1-cycle latency of the `tx_bit_stuff` module, the combinational flag signals and the current state are delayed by exactly one clock cycle using sequential registers: `flag_en_pipe` <= `flag_en_comb` `flag_d_pipe` <= `flag_d_comb` `state_pipe` <= `state`

COMBINATIONAL LOGIC (Final Output Multiplexer) *(Evaluates the delayed `state_pipe` to ensure clock synchronization with the bit stuffer output)*

- *If `state_pipe` is `TX_OPEN_FLAG` or `TX_CLOSE_FLAG`:* `tx_en_out` = `flag_en_pipe` `tx_d_out` = `flag_d_pipe`
- *Else (Payload or FCS states):* `tx_en_out` = `stuffer_en_out` (Dynamically controlled by stuffer zero-insertions) `tx_d_out` = `stuffer_d_out`

6.1.6 7. RX Framer (sdlc_rx_framer.sv)

SEQUENTIAL LOGIC (The 7-Bit CRC Shield) *(This structure prevents the final 7 bits of the closing 0x7E flag from corrupting the CRC math engine before the unstuffer realizes it's a flag).*

- **EVENT: Reset or FSM in** `HUNT` **or** `flag_detect` **/** `abort_detect` `crc_valid_count` = 0 (Empties the delay line).
- **EVENT: Valid Payload Bit** (`state == RECEIVE && destuff_en`) `crc_delay_line` shifts in `destuff_d` (MSB first internal shifting: `{destuff_d, crc_delay_line[6:1]}`). *If* `crc_delay_full` *is false:* `crc_valid_count` += 1 (Fills the pipeline). *Note:* The CRC engine is fed from `crc_delay_line[0]`, ensuring the math is exactly 7 bits "in the past".

SEQUENTIAL LOGIC (CRC Reset)

- **EVENT: Clock Edge** `flag_detect_d1 <= flag_detect` (Delays the flag pulse by 1 cycle). `crc_rst <= flag_detect_d1 || (state == HUNT)`. *Why?* This 1-cycle delay gives the main FSM the exact clock edge it needs to read the final `crc_valid` residue before the math engine is wiped clean.

SEQUENTIAL LOGIC (The Main FSM & Shadow Buffers) *(Note:* `rx_valid`*,* `rx_end_of_frame`*, and* `rx_crc_error` *always default to 0 at the start of every cycle to create single-cycle pulses).*

- **EVENT:** `abort_detect` **(Highest Priority)** Forces `state` -> `HUNT`. Drops any current frame progress.
- **EVENT:** `flag_detect` **(Frame Boundary)** *If currently in* `RECEIVE` *AND* `byte_cnt >= 2` *(Valid payload check):* `rx_end_of_frame` = 1. *If* `crc_valid` *is false:* `rx_crc_error` = 1. `state` -> `RECEIVE` (A closing flag immediately acts as the opening flag for the next frame). `bit_cnt` = 0. `byte_cnt` = 0.
- **EVENT: Valid Payload Bit** (`state == RECEIVE` **&&** `destuff_en`) `destuff_d` is shifted into `shift_reg` (LSB first). *If* `bit_cnt` *= 7 (Full Byte Assembled):* **Shadow Buffer Execution:** `byte_history_1` <- newly assembled byte. `byte_history_2` <- `byte_history_1`. `rx_data` <- `byte_history_2`. *If* `byte_cnt >= 2:* `rx_valid` = 1 (Tells CPU to grab the byte). *Why wait for byte 2?* Because the shadow buffer traps the final 2 CRC bytes of the frame. They are never validated, meaning the CPU only ever sees pure payload. `byte_cnt` += 1. `bit_cnt` = 0. *Else:* `bit_cnt` += 1.

COMBINATIONAL LOGIC / ROUTING

- `crc_delay_full` = (`crc_valid_count == 3'd7`).
- `crc_en` = (`state == RECEIVE` && `destuff_en` && `crc_delay_full`). (Only enables the CRC math engine when the 7-bit shield is fully saturated with verified payload bits).
- `rx_abort` = `abort_detect` (Passes the abort condition directly to the CPU interface).

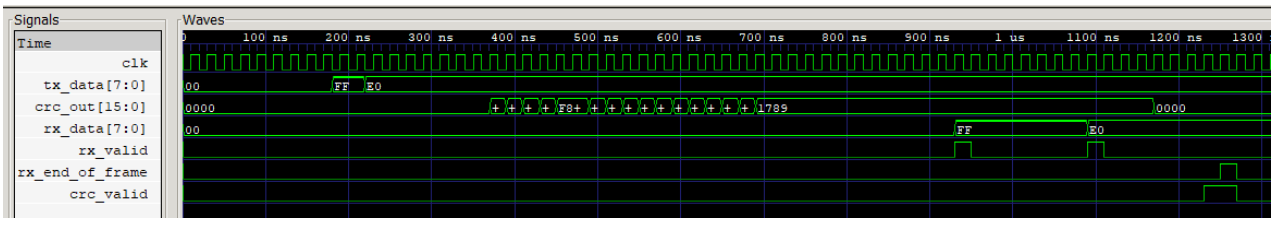


Figure 10: Functional Verification via SDLC Loopback Test

7 Conclusion

The modernization of the Ooty Radio Telescope's (ORT) Declination Control System required a foundational understanding of its undocumented legacy system. Through this project, a complete and verified pathway from legacy software analysis to modern FPGA-based hardware prototyping was successfully established.

The successful execution of this project yielded several critical deliverables that directly support the future reliability of the ORT:

- **Legacy Protocol Extraction:** By reverse-engineering the native DOS `DECMAY19.C` control software, the exact byte-level command structures for power, RF, and IF declination configurations were mapped. Furthermore, the dynamic, real-time trigonometric calculations dictating the Local Oscillator (LO) Phase Bits were documented, proving that a static look-up approach is insufficient for future modernization.
- **Custom Hardware Interfacing:** The challenge of asynchronous clock recovery on the legacy Intel 8273 multidrop bus was successfully bypassed. By integrating a 74LS123 monostable multivibrator with a TI C2000 microcontroller, a reliable, non-intrusive physical line sniffer was developed to log raw NRZI bitstreams.
- **Data Verification and Diagnostics:** Custom Python tooling enabled the complete decoding of captured SDLC frames, definitively proving the use of a reversed-polynomial CRC-CCITT (`0x8408`) implementation. Additionally, the monitoring phase isolated a specific `0xAA` telemetry anomaly within Module 23, immediately highlighting an active hardware fault to be addressed.
- **SystemVerilog Proof-of-Concept:** The theoretical and physical observations were successfully translated into a modern, modular RTL design. With the NRZI encoding, bit stuffing, CRC generation/verification, and Tx/Rx framing completely modeled and validated, the SystemVerilog SDLC controller proves that the legacy architecture can be accurately replicated.

Ultimately, this proof-of-concept eliminates the dependency on obsolete Intel 8085/8273 EoL Chips, positioning the ORT to confidently transition its N-S beam steering communications to a robust and modern architecture.

8 References

Technical Manuals & Datasheets

1. **Intel Corporation.** *Peripheral Design Handbook*. (1981).
2. **Texas Instruments.** *TMS320F2837xD Dual-Core Microcontrollers Technical Reference Manual*.
3. **IBM Corporation.** *Synchronous Data Link Control (SDLC) Concepts Manual*.

Legacy ORT Hardware Documentation

4. **DEC_MasterCard Schematic:** *SDLC Controller for IBM PC/XT/AT*. (Ooty Radio Telescope Internal Documentation)
5. **DEC_SlaveCard Schematic:** *New Array Dec. Control & Monitor System*. (Ooty Radio Telescope Internal Documentation)

Legacy ORT Software & Source Code

6. **MCR Control Software:** `DECMAY19.C` (Legacy C Source Code)
7. **MCR Control Software:** `ALL_D1.C` (Legacy C Source Code)

9 Appendix A: Python Notebook: Frame Generation & NRZI Coding

```
# %% [markdown]
# # SDLC/NRZI reference notebook
# This notebook builds one legacy SDLC transmit frame from address
    /control/payload bytes, compute FCS, bit-stuff, NRZI-encode,
    and generate a 32x oversampled waveform for comparison with
    hardware captures.

# %%
import numpy as np
import pandas as pd
from pathlib import Path
import matplotlib.pyplot as plt

OUT = Path('output')
OUT.mkdir(exist_ok=True)

# %% [markdown]
# ## 1. Build Transmit Frame
# Fill `addr`, `ctrl`, and `payload` using the final transmitted
    values from NEWDEC. For RFDEC, `ctrl=0x57`; for IFDEC (`modnum
    ==23`), `ctrl=0x97`. NEWDEC uses `modnum` as the address byte
    and `tx_data_buf[0..bytcnt-1]` as the payload.

# %%
addr = 0x79 # RF DEC: 0x01 - 0x16) 1 - 22 | IF DEC: (0x17) 23
ctrl = 0x98 # RF DEC: 0x57 | IF DEC: 0x97
payload = [ 0x07 ]
# Payload Length Byte Count RF DEC: 0x0d (13bytes) (104b) | IF DEC
    / IF DEC + HIGH DEC: 0x15 (21 bytes) (168b)

assert 0 <= addr <= 0xFF and 0 <= ctrl <= 0xFF
assert all(0 <= b <= 0xFF for b in payload)
frame_no_flags = [addr, ctrl] + payload

print(f"Frame (NO FLAG) (bin): {[format(b, '08b') for b in
    frame_no_flags]}")


# %% [markdown]
# ## 2. Compute SDLC FCS (CRC-16/IBM-SDLC)
# This cell computes the 16-bit frame check sequence over address,
    control, and information fields, then appends it little-endian
    as transmitted on the line.

# %%
def crc16_ibm_sdlc(bits):

    crc = 0xFFFF
    poly = 0x1021 # x^16 + x^12 + x^5 + 1  [ 1 (0001 0000 0010
        0001) ] -> [1 (0x1021)]

    for b in bits:
        msb = (crc >> 15) & 1
        crc = ((crc << 1) & 0xFFFF) | (b & 1)

        if msb:
            crc ^= poly


    crc ^= 0xFFFF
```

```
    return crc & 0xFFFF


fcs = crc16_ibm_sdlc(frame_no_flags)
fcs_lo = fcs & 0xFF
fcs_hi = (fcs >> 8) & 0xFF
frame_bytes = [0x7E] + frame_no_flags + [fcs_hi, fcs_lo] + [0x7E]
pd.DataFrame({
    'index': range(len(frame_bytes)),
    'byte_hex': [f'0x{b:02X}' for b in frame_bytes]
})

print(f"(FCS)) (bin): {[format(b, '08b') for b in [fcs_hi,fcs_lo]
    ]}")
print(f"(Frame Bytes)) (hex): {[f'0x{b:02X}' for b in frame_bytes
    ]}")
print(f"(Frame Bytes)) (bin): {[format(b, '08b') for b in
    frame_bytes]}")


# %% [markdown]
# ## 3. Serialize bytes LSB-first
# SDLC/HDLC controllers conventionally shift each byte least-
    significant bit first on the serial line, while flags remain
    the 0x7E bit pattern. This cell converts bytes to a serial bit
    list.

# %%
def bytes_to_bits_lsb_first(byte_list):
    bits = []
    for b in byte_list:
        bits.extend([(b >> i) & 1 for i in range(8)])
    return bits

flag_bits = [(0x7E >> i) & 1 for i in range(8)]
body_bits = bytes_to_bits_lsb_first(frame_no_flags + [fcs_lo,
    fcs_hi])

len(body_bits), flag_bits

# %% [markdown]
# ## 4. Bit-stuff between flags
# Bit stuffing is applied only between flags. After five
    consecutive 1 bits in the frame body, insert a 0.

# %%
def bitstuff(bits):
    stuffed = []
    ones = 0
    for bit in bits:
        stuffed.append(bit)
        if bit == 1:
            ones += 1
            if ones == 5:
                stuffed.append(0)
                ones = 0
        else:
            ones = 0
    return stuffed

stuffed_body_bits = bitstuff(body_bits)
full_bits = flag_bits + stuffed_body_bits + flag_bits
print('Unstuffed body bits:', len(body_bits))
```

```
print('Stuffed body bits  :', len(stuffed_body_bits))
print('Full frame bits    :', len(full_bits))

# %%
def bits_to_bytes(bits):
    assert len(bits) % 8 == 0
    out = []
    for i in range(0, len(bits), 8):
        byte = 0
        for b in bits[i:i+8]:
            byte = (byte << 1) | (b & 1)
        out.append(byte)
    return out
frame_bytes = bits_to_bytes(full_bits)
print("(Frame bytes) (hex):", [f"{b:02X}" for b in frame_bytes])
print("(Frame bytes) (bin):", [f"{b:08b}" for b in frame_bytes])

# %% [markdown]
# ## 5. NRZI encode
# For SDLC NRZI, data 0 causes a transition and data 1 causes no
    transition. The initial line level is arbitrary for comparison
    as long as decode uses the same convention.

# %%
def nrzi_encode(bits, initial_level=1):
    level = initial_level
    out = []
    for bit in bits:
        if bit == 0:
            level ^= 1
        out.append(level)
    return out

nrzi_bits = nrzi_encode(full_bits, initial_level=1)
pd.DataFrame({
    'bit_index': range(min(64, len(full_bits))),
    'sdlc_bit': full_bits[:64],
    'nrzi_level': nrzi_bits[:64],
})


# %% [markdown]
# ## 6. Generate 32x oversampled waveform
# The legacy link uses a 32x clock with DPLL-derived bit timing.
    This cell expands each bit into 32 samples so you can compare
    against a capture indexed by the 32x clock.

# %%
bit_rate = 9600
oversample = 32
sample_rate = bit_rate * oversample

samples = np.repeat(nrzi_bits, oversample)
t = np.arange(len(samples)) / sample_rate
wave_df = pd.DataFrame({
    'sample_idx': np.arange(len(samples)),
    'time_s': t,
    'txd_level': samples.astype(int),
})
wave_df.head(20)

# %% [markdown]
# ## 7. Save NZRI Waveform sampled with 32xCLK (DPLL)
```

```
#

# %%
frame_df = pd.DataFrame({
    'index': range(len(frame_bytes)),
    'byte_hex': [f'0x{b:02X}' for b in frame_bytes],
    'byte_dec': frame_bytes,
})
bits_df = pd.DataFrame({
    'bit_index': range(len(full_bits)),
    'sdlc_bit': full_bits,
    'nrzi_level': nrzi_bits,
})
frame_df.to_csv(OUT / 'frame_bytes.csv', index=False)
bits_df.to_csv(OUT / 'frame_bits_nrzi.csv', index=False)
wave_df.to_csv(OUT / 'frame_waveform_32x.csv', index=False)
print(OUT / 'frame_bytes.csv')
print(OUT / 'frame_bits_nrzi.csv')
print(OUT / 'frame_waveform_32x.csv')

# %%

df = pd.read_csv(OUT / 'frame_waveform_32x.csv')

plt.figure(figsize=(10, 4))
plt.step(df['time_s'], df['txd_level'], where='post')  # staircase
    , NRZI-style
plt.xlabel('Time (s)')
plt.ylabel('TXD level')
plt.title('Frame waveform (32x oversampled)')
plt.grid(axis='y', linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()

# %%
```

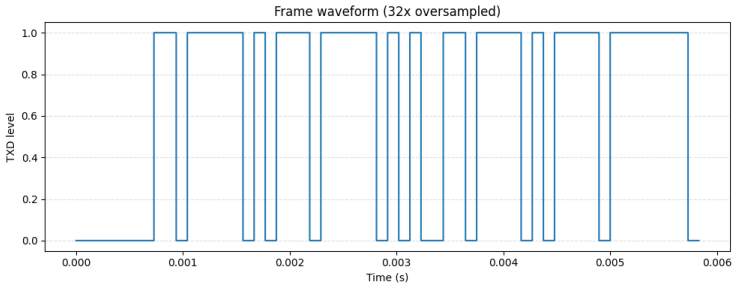


Figure 11: Synthetic NRZI Data Frame

10 Appendix B: Hardware Monitor Code

TI C2000 - F28379D LaunchXL Dev Board Code

```
//###########################################################################
// FILE:DEC_FrameMonitor.c
// TITLE:SDLC raw bit logger on F28379D.
// AUTHOR: Jayanth Krish
//###########################################################################


#include "F28x_Project.h"


#pragma DATA_SECTION(data_buf, "MYBUF");


#define BLINKY_LED_GPIO 31
#define SDLC_DATA_GPIO  123 //18
#define SDLC_DPLL_GPIO 61  //19
#define DEBUG_GPIO      122 //17


#define PKT_SYNC1       0xAA
#define PKT_SYNC2       0x55
#define PKT_TYPE_SNIFF  0x01    // Tx: 0x01 Rx: 0x02


#define BUF_LEN 5120



volatile Uint16 sample_seen = 0;


volatile Uint16 data_buf[BUF_LEN];
volatile Uint16 snapshot[BUF_LEN];


volatile Uint16 data_count = 0;
volatile Uint16 snapshot_len = 0;



volatile Uint16 block_ready = 0;
volatile Uint16 capture_done = 0;
volatile Uint16 packet_sent = 0;
volatile Uint16 sniff_active = 0;




volatile Uint16 bit_count = 0;
volatile Uint16 work_byte = 0;



__interrupt void xint1_isr(void);
__interrupt void xint2_isr(void);


void scia_init(void);
```

```
void scia_xmit(Uint16 a);
void send_packet(Uint16 type, Uint16 part, const volatile Uint16 *
    buf, Uint16 len);


void clear_capture_buffer(void);


void main(void)
{
    Uint16 part = 1;
    Uint16 len = 0;
    Uint16 i;


    InitSysCtrl();
    InitGpio();


    // LED
    GPIO_SetupPinMux(BLINKY_LED_GPIO, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(BLINKY_LED_GPIO, GPIO_OUTPUT,
        GPIO_PUSHPULL);
    GPIO_WritePin(BLINKY_LED_GPIO, 1);   // OFF initially


    // SDLC input pin
    GPIO_SetupPinMux(SDLC_DATA_GPIO, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(SDLC_DATA_GPIO, GPIO_INPUT, GPIO_ASYNC);


    // DPLL
    GPIO_SetupPinMux(SDLC_DPLL_GPIO, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(SDLC_DPLL_GPIO, GPIO_INPUT, GPIO_ASYNC);

    // Debug
    GPIO_SetupPinMux(DEBUG_GPIO, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(DEBUG_GPIO, GPIO_OUTPUT, GPIO_PUSHPULL);
    GPIO_WritePin(DEBUG_GPIO, 0);


    // USB-UART LINES
    GPIO_SetupPinMux(43, GPIO_MUX_CPU1, 15);
    GPIO_SetupPinOptions(43, GPIO_INPUT, GPIO_ASYNC);


    GPIO_SetupPinMux(42, GPIO_MUX_CPU1, 15);
    GPIO_SetupPinOptions(42, GPIO_OUTPUT, GPIO_PUSHPULL);


    DINT;
    InitPieCtrl();
    IER = 0x0000;
    IFR = 0x0000;
    InitPieVectTable();


    EALLOW;
    PieVectTable.XINT1_INT  = &xint1_isr; //DPLL
    PieVectTable.XINT2_INT = &xint2_isr;  // data-line edge
    EDIS;
```

```
    // DPLL on GPIO123 to XINT1
    GPIO_SetupXINT1Gpio(SDLC_DPLL_GPIO);
    XintRegs.XINT1CR.bit.POLARITY = 1;   // 0: falling edge 1:
        rising edge
    XintRegs.XINT1CR.bit.ENABLE   = 1;


    // data line on XINT2, trigger on falling edge (idle->active)
    GPIO_SetupXINT2Gpio(SDLC_DATA_GPIO);
    XintRegs.XINT2CR.bit.POLARITY = 0; // falling
    XintRegs.XINT2CR.bit.ENABLE   = 1;



    PieCtrlRegs.PIECTRL.bit.ENPIE = 1;
    PieCtrlRegs.PIEIER1.bit.INTx4 = 1;   // XINT1
    PieCtrlRegs.PIEIER1.bit.INTx5 = 1;   // XINT2
    IER |= M_INT1;


    EINT;
    ERTM;


    // Initialize SCI USB-UART Interface
    scia_init();


    clear_capture_buffer();


    for(;;)
    {
        if(sample_seen)
        {
            //DEBUG

            GPIO_WritePin(BLINKY_LED_GPIO, 0);
            GPIO_WritePin(BLINKY_LED_GPIO, 1);


            GPIO_WritePin(DEBUG_GPIO, 1);
            DELAY_US(10);
            GPIO_WritePin(DEBUG_GPIO, 0);

            sample_seen = 0;
        }


        if(block_ready)
        {
            DINT;
            len = snapshot_len;
            for(i = 0; i < len; i++)
            {
                snapshot[i] = data_buf[i] & 0x00FF;
            }
            block_ready = 0;
            EINT;


            if(len > 0)
            {
```

```
                send_packet(PKT_TYPE_SNIFF, part, snapshot, len);
                packet_sent = 1;
            }
        }
    }
}


__interrupt void xint1_isr(void)
{
    Uint16 level;
    Uint16 bit;


    if(!sniff_active || capture_done)
    {
        PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
        return;
    }

    DELAY_US(10);

    level = GPIO_ReadPin(SDLC_DATA_GPIO);
    bit   = (level != 0U) ? 1U : 0U;

    sample_seen = 1;

    work_byte = (work_byte << 1) | bit;
    bit_count++;


    if(bit_count == 8)
    {
        if(data_count < BUF_LEN)
        {
            data_buf[data_count] = work_byte & 0x00FF;
            data_count++;
        }


        work_byte = 0;
        bit_count = 0;


        if(data_count >= BUF_LEN)
        {
            capture_done = 1;
            snapshot_len = BUF_LEN;
            block_ready = 1;
            sniff_active = 0;
        }
    }



    PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
}


__interrupt void xint2_isr(void)   // SDLC data edge (idle->active
    )
{
    if(!sniff_active && !capture_done)
```

```
    {
        sniff_active     = 1;
        work_byte        = 0;
        bit_count        = 0;
        data_count       = 0;
        block_ready      = 0;
        snapshot_len     = 0;
    }


    PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
}


void scia_init(void)
{
    // 1 stop bit, no parity, 8-bit character, async, idle-line
    SciaRegs.SCICCR.all = 0x0007;


    // enable TX, RX, internal SCI clock
    SciaRegs.SCICTL1.all = 0x0003;


    // no SCI interrupts needed
    SciaRegs.SCICTL2.all = 0x0000;


    // These baud values are from TI F2837xD SCI echoback-style
        examples
    SciaRegs.SCIHBAUD.all = 0x0002;
    SciaRegs.SCILBAUD.all = 0x008B;


    // optional FIFO init
    SciaRegs.SCIFFTX.all = 0xE040;
    SciaRegs.SCIFFRX.all = 0x2044;
    SciaRegs.SCIFFCT.all = 0x0000;


    // release SCI from reset
    SciaRegs.SCICTL1.all = 0x0023;
}


void scia_xmit(Uint16 a)
{
    while(SciaRegs.SCIFFTX.bit.TXFFST != 0)
    {
    }


    SciaRegs.SCITXBUF.all = (a & 0x00FF);
}


void send_packet(Uint16 type, Uint16 part, const volatile Uint16 *
    buf, Uint16 len)
{
    Uint16 i;


    scia_xmit(PKT_SYNC1);
```

```
    scia_xmit(PKT_SYNC2);
    scia_xmit(type);
    scia_xmit((part >> 8) & 0x00FF);
    scia_xmit(part & 0x00FF);
    scia_xmit((len >> 8) & 0x00FF);
    scia_xmit(len & 0x00FF);


    for(i = 0; i < len; i++)
    {
        scia_xmit(buf[i] & 0x00FF);
    }
}


void clear_capture_buffer(void)
{
    Uint16 i;


    bit_count = 0;
    work_byte = 0;
    data_count = 0;
    block_ready = 0;
    snapshot_len = 0;


    for(i = 0; i < BUF_LEN; i++)
    {
        data_buf[i] = 0;
    }
}
```

Python Serial Monitor

```
from pathlib import Path
import argparse
import serial

SYNC1 = 0xAA
SYNC2 = 0x55

TYPE_TX = 0x01
TYPE_RX = 0x02


def read_exact(ser, n: int) -> bytes:
    data = bytearray()
    while len(data) < n:
        chunk = ser.read(n - len(data))
        if not chunk:
            raise TimeoutError(f"Timed out waiting for {n} bytes,
                got {len(data)}")
        data.extend(chunk)
    return bytes(data)


def find_sync(ser) -> None:
    while True:
        b = read_exact(ser, 1)[0]
        if b != SYNC1:
            continue
        b2 = read_exact(ser, 1)[0]
        if b2 == SYNC2:
            return


def type_name(pkt_type: int) -> str:
    if pkt_type == TYPE_TX:
        return "tx"
    if pkt_type == TYPE_RX:
        return "rx"
    return f"unknown_{pkt_type:02x}"


def main():
    parser = argparse.ArgumentParser(description="Receive sniffer
        chunks from C2000 over UART")
    parser.add_argument("--port", required=True, help="Serial port
        , e.g. COM14")
    parser.add_argument("--baud", type=int, default=9600, help="
        Baud rate")
    parser.add_argument("--outdir", default="captures", help="
        Output directory")
    args = parser.parse_args()

    outdir = Path(args.outdir)
    outdir.mkdir(parents=True, exist_ok=True)

    with serial.Serial(args.port, args.baud, timeout=2) as ser:
        print(f"Listening on {ser.port} at {ser.baudrate} baud")
        print(f"Writing files to: {outdir.resolve()}")

        while True:
            try:
```

```
                find_sync(ser)

                header = read_exact(ser, 5)
                pkt_type = header[0]
                part = (header[1] << 8) | header[2]
                length = (header[3] << 8) | header[4]

                payload = read_exact(ser, length)

                name = f"bytestream.part{part:04d}.{type_name(
                    pkt_type)}.bin"
                path = outdir / name
                path.write_bytes(payload)

                print(f"saved {name} ({length} bytes)")
            except KeyboardInterrupt:
                print("\nStopped.")
                break
            except Exception as e:
                print(f"warning: {e}")


if __name__ == "__main__":
    main()
```

11 Appendix C: Python Notebook: Process captured NRZI Data

(Access at this link: SDLC NRZI Python Notebook)

```
# %%
# Imports
import csv
from pathlib import Path
import yaml2plot as y2p

# %%
# HELPER FUNCTIONS
def read_column_bits(path, column_name):
    bits = []
    with Path(path).open("r", newline="") as f:
        r = csv.DictReader(f)
        for row in r:
            bits.append(int(row[column_name]))
    return bits

# %%
# Binary data --> CSV

input_file = Path("data/rx_35__trial1.bin")
f_name= input_file.stem
data = input_file.read_bytes()

raw_levels = []
for b in data:
    for i in range(7, -1, -1):   # MSB first
        raw_levels.append((b >> i) & 1)

# RAW LEVELS
raw_output_file = Path(f"raw_levels/raw_levels_{f_name}.csv")
with raw_output_file.open("w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["sample_index", "raw_level"])
    for i, lvl in enumerate(raw_levels):
        w.writerow([i, lvl])

print(f"RAW: Wrote {len(raw_levels)} samples to {raw_output_file}"
    )

# %%
# Generate Plot - Raw Levels

plt_fname=f"raw_levels/raw_levels_{f_name}.csv"

data = read_column_bits(plt_fname, "raw_level")

plot_data = {
    "sample_index": list(range(len(data))),
    "bit": data,
}

spec = y2p.PlotSpec.from_yaml(f"""
title: "{plt_fname}"

X:
  signal: "sample_index"
  label: "Bit index"
  log_scale: false
```

```
Y:
  - label: "Bit value"
    signals:
      Data: "bit"

height: 500
width: 1000
show_rangeslider: true
""")

fig = y2p.plot(plot_data, spec)

# %%
# SHOW FIGURE
#fig.update_yaxes(range=[-0.2, 1.2])
fig.update_traces(line_shape="hv")
fig.show()

# %% [markdown]
# # CSV Plotter

# %%
# Generate Plot - Raw Levels
f_plot="jun_15_tx"

plt_fname=f"raw_levels/raw_levels_{f_plot}.csv"

data = read_column_bits(plt_fname, "raw_level")

plot_data = {
    "sample_index": list(range(len(data))),
    "bit": data,
}

spec = y2p.PlotSpec.from_yaml(f"""
title: "{plt_fname}"

X:
  signal: "sample_index"
  label: "Bit index"
  log_scale: false

Y:
  - label: "Bit value"
    signals:
      Data: "bit"

height: 500
width: 1000
show_rangeslider: true
""")

fig_plt = y2p.plot(plot_data, spec)

# %%
# SHOW FIGURE
#fig.update_yaxes(range=[-0.2, 1.2])
fig_plt.update_traces(line_shape="hv")
fig_plt.show()

# %% [markdown]
# # CRC-CCIT Implementation
```

```
# %%
# CRC Functions
def update_sdlc_crc(crc, ch):
    """Processes a single byte for standard SDLC/HDLC CRC (LSB-
       first)."""
    # Poly 0x1021 reversed for LSB-first processing is 0x8408
    POLY_REVERSED = 0x8408

    crc ^= ch
    for _ in range(8):
        if crc & 1:
            crc = (crc >> 1) ^ POLY_REVERSED
        else:
            crc >>= 1
    return crc & 0xFFFF

def calculate_sdlc_crc(hex_string):
    """
    Calculates the true SDLC CRC and the bit-reversed Sniffer
       display format.
    """
    clean_hex = hex_string.replace(" ", "").replace("0x", "")
    data_bytes = bytes.fromhex(clean_hex) if clean_hex else b""

    crc = 0xFFFF # SDLC Initial value

    for byte in data_bytes:
        crc = update_sdlc_crc(crc, byte)

    # Final XOR for SDLC FCS
    crc ^= 0xFFFF

    # The sniffer reads the low byte first off the wire
    sniffed_fcs = int('{:016b}'.format(crc)[::-1], 2)

    return {
        "hex_input": clean_hex.upper(),
        "text_length": len(data_bytes),
        "true_crc": f"0x{crc:04X}",
        "sniffer_fcs": f"{sniffed_fcs:04X}"
    }

def print_crc_results(hex_string, description=""):
    """Utility function to cleanly print the results in a notebook
       ."""
    res = calculate_sdlc_crc(hex_string)
    label = f" ({description})" if description else ""
    print(f"--- Results for input: '{res['hex_input']}'{label} ---
       ")
    print(f"True CRC: {res['true_crc']}")
    print(f"Hardware Sniffer FCS:  {res['sniffer_fcs']}")
    print(f"Length:                {res['text_length']} bytes\n")


# %%
# CRC TEST
# Addr: 0xFF Ctrl: 0xE0 I-Field: Nil
# 0001 0111 1000 1001
# 1001 0001 1110 1000
print_crc_results("FFE0", description="PWR OFF")

# %%
```

```
# CRC TEST
# Addr: 0x00 Ctrl: 0x00 I-Field: 255*0x00
print_crc_results(257*"00", description="PWR ON")

# %%
# CRC TEST
print_crc_results("0157E483562FB9E4835A2FB9E48313", description="
    RF DEC MOD1")

# %%
# CRC TEST
print_crc_results("0257E483562FB9E4835A2FB9E4830A", description="
    RF DEC MOD2")

# %%
# CRC TEST
print_crc_results("01570000000000000000000000000A", description="
    RF DEC 00 00 00 MOD1")

# %%
# CRC TEST | IF DELAY SET
print_crc_results("179789271BDB4B606D500D4BB558674D03F9FF7F020000"
    , description="IF DEC")

# %%
# CRC TEST | TELEMETRY RX DATA (IF)
print_crc_results("FF97"+42*"AA", description="RX DATA IF")
```

```
                CRC Results
-----------------------------------------
  COMMAND |   A+C+I   |  CRC    | FCS (H/W)
-----------------------------------------
  PWR ON  | 257*"00" | 0x58F2 | 0x4F1A
  PWR OFF |   FFE0   | 0x1789 | 0x91E8
```

12 Appendix D: SystemVerilog RTL SDLC Controller

(Access at this link: SDLC Controller RTL)

Design Sources

```
rtl/nrzi_encode.sv
rtl/nrzi_decode.sv
rtl/crc_ccitt_fcs.sv
rtl/tx_bit_stuff.sv
rtl/rx_bit_unstuff.sv
rtl/sdlc_tx_framer.sv
rtl/sdlc_rx_framer.sv
```

Testbenches

```
tb/tb_sdlc_loopback.sv
tb/tb_crc_ccitt_fcs.sv
tb/tb_nrzi_decode.sv
tb/tb_nrzi_encode.sv
tb/tb_nrzi_loopback.sv
tb/tb_rx_bit_unstuff.sv
tb/tb_sdlc_rx_framer.sv
tb/tb_sdlc_tx_framer.sv
tb/tb_tx_bit_stuff.sv
```